Cambridge (CIE) A Level Computer Science

Your notes

Data Definition Language (DDL) & Data Manipulation Language (DML)

Contents

- SQL

Your notes

DDL & DML

What is DDL & DML?

- Data Definition Language (**DDL**) is used to **build and manage the structure** of a relational database
- It defines **how data is stored** by creating and modifying tables, fields, indexes, and relationships
- Data Manipulation Language (**DML**) is used to **manage the data** stored within the database structures created by DDL
- It deals with **adding, updating, deleting**, and **retrieving** data
- Both DDL and DML use **SQL syntax** to interact with the database
- DDL **structures** the database, while DML maintains the **contents** of the database
- Most modern **DBMSs use SQL** to support both DDL and DML operations

Basic DDL commands

Command	Purpose	Example
CREATE DATABASE	Creates a new database	CREATE DATABASE SchoolDB
CREATE TABLE	Creates a new table with specified fields and data types	CREATE TABLE Students (StudentID INTEGER, Name VARCHAR(50))
Data types for attributes:	Defines the type of data each field can store	
CHARACTER	Fixed-length text	Name CHARACTER(10)
VARCHAR(n)	Variable-length text (max length n)	Email VARCHAR(100)
BOOLEAN	True or False value	IsActive BOOLEAN
INTEGER	Whole numbers	Age INTEGER
REAL	Decimal numbers	Score REAL

© 2025 Save My Exams, Ltd.

DATE	Calendar date	DateOfBirth DATE
TIME	Time of day	StartTime TIME
ALTER TABLE	Changes an existing table (e.g. add or remove fields)	ALTER TABLE Students ADD Email VARCHAR(100)
PRIMARY KEY (field)	Sets a field as the primary key (unique identifier)	PRIMARY KEY (StudentID) inside a CREATE TABLE or ALTER TABLE command
FOREIGN KEY (field) REFERENCES Table(Field)	Sets up a foreign key relationship between tables	FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)

Your notes

Example: School database

- You are designing a database to store information about **students** and the **courses** they are enrolled in
- Each student can enrol in many courses, and each course can have many students (a **many-to-many relationship**)
- Three tables are needed:
  - Students – to store student details
  - Courses – to store course details
  - Enrolments – a link table to handle the many-to-many relationship

```
-- Create the Students table
CREATE TABLE Students (
   StudentID INTEGER,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   DateOfBirth DATE,
   PRIMARY KEY (StudentID)
);

-- Create the Courses table
CREATE TABLE Courses (
   CourseID INTEGER,
   CourseName VARCHAR(100),
   Credits INTEGER,
   PRIMARY KEY (CourseID)
);

-- Create the Enrolments table to link Students and Courses
CREATE TABLE Enrolments (
   StudentID INTEGER,
   CourseID INTEGER,
   EnrolDate DATE,
   PRIMARY KEY (StudentID, CourseID),
   FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
```

```
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

Your notes

Basic DML commands

Retrieving data

Command	Purpose	Example
SELECT ... FROM	Retrieves specific columns from a table	SELECT FirstName, LastName FROM Students
WHERE	Filters results based on a condition	SELECT * FROM Students WHERE Age > 16
ORDER BY	Sorts the results in ascending or descending order	SELECT * FROM Students ORDER BY LastName ASC
GROUP BY	Groups rows that have the same value in specified columns	SELECT CourseID, COUNT(*) FROM Enrolments GROUP BY CourseID
INNER JOIN	Combines rows from two tables where there is a matching key	SELECT Students.FirstName, Courses.CourseName FROM Students INNER JOIN Enrolments ON Students.StudentID = Enrolments.StudentID INNER JOIN Courses ON Enrolments.CourseID = Courses.CourseID
SUM()	Calculates the total of a numeric column	SELECT SUM(Credits) FROM Courses
COUNT()	Counts the number of rows (or non-null entries in a column)	SELECT COUNT(*) FROM Enrolments
AVG()	Calculates the average value of a numeric column	SELECT AVG(Age) FROM Students

Data maintenance

Command	Purpose	Example
INSERT INTO	Adds a new record to a table	INSERT INTO Students (StudentID, FirstName, LastName) VALUES (1, 'Ali', 'Khan')

© 2025 Save My Exams, Ltd.

DELETE FROM	Removes records from a table	DELETE FROM Students WHERE StudentID = 1
UPDATE	Modifies existing data in a table	UPDATE Students SET Age = 17 WHERE StudentID = 1

Your notes

Example: School database

- You're working with a school database containing three tables:
  - Students(StudentID, FirstName, LastName, Age)
  - Courses(CourseID, CourseName, Credits)
  - Enrolments(StudentID, CourseID, EnrolDate)
- You want to **query the database** and **maintain the data** as students enrol in courses, update their details, or leave the school

Querying

```
-- 1. Get the first and last names of all students
SELECT FirstName, LastName FROM Students;

-- 2. List students who are older than 16
SELECT * FROM Students WHERE Age > 16;

-- 3. List all students sorted alphabetically by surname
SELECT * FROM Students ORDER BY LastName ASC;

-- 4. Count how many students are enrolled on each course
SELECT CourseID, COUNT(*) FROM Enrolments GROUP BY CourseID;

-- 5. Show student names with the courses they are enrolled on
SELECT Students.FirstName, Students.LastName, Courses.CourseName
FROM Students
INNER JOIN Enrolments ON Students.StudentID = Enrolments.StudentID
INNER JOIN Courses ON Enrolments.CourseID = Courses.CourseID;

-- 6. Get the total number of course credits available
SELECT SUM(Credits) FROM Courses;

-- 7. Get the average age of all students
SELECT AVG(Age) FROM Students;
```

Maintaining

```
-- 1. Add a new student to the Students table
INSERT INTO Students (StudentID, FirstName, LastName, Age)
VALUES (4, 'Amira', 'Patel', 17);

-- 2. Enrol that student on CourseID 2
INSERT INTO Enrolments (StudentID, CourseID, EnrolDate)
VALUES (4, 2, '01/09/2025'); -- UK date format (DD/MM/YYYY)

-- 3. Update a student's age
UPDATE Students SET Age = 18 WHERE StudentID = 4;
```


© 2025 Save My Exams, Ltd.

```
-- 4. Delete a student who has left the school
DELETE FROM Students WHERE StudentID = 4;
```

Your notes

Worked Example

Part of a database table BIRD_TYPE is shown:

BirdID	Name	Size
0123	Blackbird	Medium
0035	Jay	Large
0004	Raven	Large
0085	Robin	Small

The database only supports these data types:

- character
- varchar
- Boolean
- integer
- real
- date
- time

Write a Structured Query Language (SQL) script to define the table Bird_Type. **[4]**

Answer

- CREATE TABLE start and end bracket [1 mark]
- Bird ID as CHAR/VARCHAR [1 mark]
- Name **and** size as VARCHAR/CHAR [1 mark]
- Bird ID as primary key [1 mark]

Example answer

```
CREATE TABLE BIRD_TYPE(
BirdID CHAR(4) NOT NULL,
Name VARCHAR(9),
Size VARCHAR(6),
PRIMARY KEY (BirdID)
);
```